

TradeLog

Trading Journal Web Application — Full Project Overview, Architecture and Request Flow

React 19

TypeScript

Vite 8

Tailwind CSS

Express 5

Mongoose 9

MongoDB

JWT Auth

1. What the Project Is

TradeLog is a trading journal web application. A trader records every trade (entry/exit prices, stop loss, target, pair, session, direction, result, notes, screenshots, psychology) and then reviews statistics and performance through dashboards, analytics, a rules checklist and backtest tools.

The application is a **frontend-backend separated** app:

- **Frontend** (frontend/) — React single-page application (SPA).
- **Backend** (backend/) — REST API built with Express, exposing JWT-authenticated endpoints.
- **Database** — MongoDB (Mongoose ODM). Each user's data is scoped and isolated from every other user.

2. High-Level Architecture

The backend is the **single source of truth**. The frontend talks to it over HTTP/JSON.

Layer	What it does	Key files
Browser / SPA	Renders UI, holds the JWT, keeps local UI state (theme, settings).	frontend/src/
API client	Wraps <code>fetch</code> , attaches the <code>Bearer</code> token, handles 401.	frontend/src/lib/api.ts
Proxy	Vite dev server forwards <code>/api/*</code> to <code>localhost:5000</code> (no CORS in dev).	frontend/vite.config.ts
Backend API	Express 5 server: routing, auth middleware, validation, controllers, error handling.	backend/server.js , backend/routes/ , backend/controllers/ , backend/middleware/
Data layer	Mongoose models for <code>User</code> and <code>Trade</code> , connection setup.	backend/models/ , backend/config/db.js
Database	MongoDB — local instance or MongoDB Atlas cluster.	—



3. Project Layout

```

/workspace
+-- README.md           Project overview & getting started
+-- backend/           Express REST API
| +-- server.js         App entry: middleware, routes, DB connect, startup
| +-- .env.example      Documented environment variables (MONGO_URI, JWT_SECRET...)
| +-- config/db.js      Mongoose connection (reads MONGO_URI)
| +-- models/User.js    User schema (bcrypt hash, toJSON hides passwordHash)
| +-- models/Trade.js   Trade schema (string _id = client UUID, scoped to user)
| +-- routes/index.js   Mounts /health, /auth, /trades
| +-- routes/auth.routes.js  register, login, me, profile, password

```

```

| +-- routes/tradeRoutes.js      CRUD + bulk + clear-all (all protected)
| +-- routes/health.routes.js   GET /api/health
| +-- controllers/authController.js  register/login/me/updateProfile/changePassword
| +-- controllers/tradeController.js list/get/create/update/delete/clearAll/bulk
| +-- middleware/auth.js        JWT "protect" middleware
| +-- middleware/validateTrade.js Shape/format validation for trade payloads
| +-- middleware/errorHandler.js  notFound + centralized error handler
| +-- utils/asyncHandler.js      Wraps async controllers, forwards errors to next()
+-- frontend/                   React SPA
| +-- index.html / vite.config.ts / tailwind.config.js / package.json
| +-- src/main.tsx              React root + provider hierarchy
| +-- src/App.tsx               Router + protected routes + layout
| +-- src/types/Trade.ts        Frontend Trade type (mirrors backend schema)
| +-- src/lib/api.ts            API client (auth + trades endpoints)
| +-- src/lib/authStore.ts / AuthProvider.tsx  JWT session management
| +-- src/lib/TradeProvider.tsx / TradeContext.tsx  Server-synced trade state
| +-- src/lib/csv.ts            CSV/JSON export + import (TradeLog & broker formats)
| +-- src/lib/analytics.ts       Win rate, RR, profit stats
| +-- src/pages/                Dashboard, TradeLog, AddEditTrade, AnalyticsPage,
|                               RulesChecklist, BacktestProgress, ChartBacktest,
|                               Login, Profile
| +-- src/components/           Navbar, TradeForm, TradeTable, StatCard, charts,
|                               ThemeToggle, BackupReminder

```

4. Tech Stack Summary

Frontend

React 19, Vite 8, TypeScript ~6, React Router 7, Tailwind CSS 3, Recharts 3, lucide-react icons

Backend

Node.js ≥ 20, Express 5, Mongoose 9, JSON Web Tokens (`jsonwebtoken`), `bcryptjs` , `helmet` , `cors` , `dotenv`

Database

MongoDB 8 (local in this sandbox, or MongoDB Atlas from your machine)

Dev tooling

Vite dev server with `/api` proxy, ESLint (typescript-eslint + react-hooks), TypeScript `tsc -b` build, Tailwind + PostCSS, Autoprefixer

5. Provider Hierarchy (Frontend)

Wrapping order in `frontend/src/main.tsx` — inner providers can use outer ones:

```

<ThemeProvider>      -- dark / light theme (persisted in localStorage)
  <AuthProvider>      -- user session, JWT storage, login/register/profile
    <SettingsProvider> -- backtest goal settings, backup-reminder counters
      <TradeProvider>  -- trade list + CRUD sync with the backend
        <App />        -- Router + pages

```

6. Request Flow — End to End

Every screen request follows the same path:

1. Browser component calls a hook (`useAuth()` , `useTrades()`).
2. Hook calls the API client `frontend/src/lib/api.ts` .
3. `api.ts` reads the saved JWT and calls `fetch("/api/...")` with `Authorization: Bearer <token>` .
4. Vite dev server proxies `/api/*` → `http://localhost:5000` .
5. Express middleware chain: `helmet()` → `cors()` → `express.json()` → router → route → `protect` (JWT) → controller → Mongoose → MongoDB.
6. JSON response returns; on `401` the API client clears the token (auto-logout).
7. Component updates state; Recharts / tables re-render.

- 1 **Component** — e.g. `TradeLog` calls `const { trades, addTrade } = useTrades()` .
- 2 **Context** — `TradeProvider` runs the optimistic update and calls the client.
- 3 **API client** — `api.trades.create(trade)` → `fetch('/api/trades', { method: 'POST', headers: { Authorization: 'Bearer ...' } })` .
- 4 **Vite proxy** — rewrites the request to `http://localhost:5000/api/trades` .
- 5 **Backend** — `tradeRoutes.js` → `protect` verifies JWT and loads `req.user` → `validateTrade` checks the payload → `TradeController.createTrade`

```
tradecontroller.createTrade.
```

- 6 **Mongoose** — `Trade.create({ user: req.user._id, ... })` inserts into MongoDB.
- 7 **Response** — `201 { success, message, data: trade }` returns; `TradeProvider` replaces the optimistic record with the server-confirmed one.

7. Authentication Flow

7.1 Registration

- 1 User fills `Login.tsx` register form (name, email, password).
- 2 `AuthProvider.register()` → `api.auth.register()` → `POST /api/auth/register`.
- 3 Backend validates (email regex, password $\geq$ 6 chars). Duplicate email → `409`.
- 4 Password hashed with `bcrypt.hash(password, 10)`; only the hash is stored.
- 5 User saved; a JWT is signed: `jwt.sign({ id }, JWT_SECRET, { expiresIn: '7d' })`.
- 6 Response: `201 { success, token, data: user }`. Frontend stores the token and sets the session.

7.2 Login & “Remember me”

- 1 `POST /api/auth/login` with `{ email, password }`.
- 2 Backend finds the user by email, compares with `bcrypt.compare()`. Wrong credentials → `401`.
- 3 Success returns a JWT. Frontend stores it:
 - **Remember me = on** → `localStorage` (persists across browser restarts).
 - **Remember me = off** → `sessionStorage` (cleared when the tab closes).
- 4 `AuthProvider` sets `user`; protected routes now render.

7.3 Session restore (page refresh)

- 1 On mount, `AuthProvider` checks for a stored token (`getAuthToken()`).
- 2 If found, it calls `GET /api/auth/me` (JWT in the header) to fetch the user profile.
- 3 Valid token → session restored. Invalid/expired → token cleared, user redirected to `/login`.

7.4 Profile update & password change

- `PUT /api/auth/profile` — protected; updates the logged-in user's `name`.
- `PUT /api/auth/password` — protected; verifies the current password, re-hashes and stores the new one.

Security notes: the `toJSON` transform on the User model strips `passwordHash` from every API response; passwords are only ever stored as bcrypt hashes; the backend refuses to start without a `JWT_SECRET`; `helmet` sets secure HTTP headers.

8. Trade Management Flow

All trade endpoints are behind `protect`. Every query filters by `user: req.user._id`, so **each account only ever sees and modifies its own trades** (server-side isolation).

8.1 Trade fields (shared schema)

The frontend type `frontend/src/types/Trade.ts` mirrors the backend `models/Trade.js` exactly: `id`, `date` (YYYY-MM-DD), `entryTime`, `pair`, `session` (London / NewYork / Overlap / Other), `direction` (Buy / Sell), `zoneType` (Supply / Demand), `entry`, `stopLoss`, `target`, `result` (Win / Loss / BreakEven / Open), `rr`, `notes`, `tags[]`, `mindset { before, after }`, `emotion`, `reason`, `screenshot`, `isValidRuleTrade`.

Trade `_id`s are **client-generated UUIDs** stored as strings. This keeps CSV/JSON import/export round-trips stable (an imported trade keeps its original id).

8.2 Creating a trade

- 1 `TradeProvider.addTrade(trade)` generates `crypto.randomUUID()` and applies the change **optimistically** (instant UI).
- 2 `POST /api/trades` → `validateTrade` → `Trade.create({ user: req.user._id, ...trade })`.
- 3 On success the server-confirmed trade replaces the optimistic one. On failure the optimistic trade is rolled back and an error banner shows.

8.3 Updating / deleting

- `PUT /api/trades/:id` — optimistic in-place update; reverted if the server rejects it.
- `DELETE /api/trades/:id` — optimistic removal; kept for undo, restored if the request fails.
- `DELETE /api/trades` — clear all trades for the current user.
- **Undo** — deleted trades are remembered in `lastDeleted`; `undoDelete()` re-imports them via `POST /api/trades/bulk`.

8.4 Bulk import (CSV / JSON)

- 1 User drops a file. `csv.ts` detects the format:
 - **TradeLog export** (has `id`, `result`, `rr` columns) → `parseCsv`.
 - **Broker export** (date/symbol/entry...) → `parseBrokerCsv` auto-maps columns and computes `rr/result`.
 - **JSON** → array of trades, or `{ trades: [...] }`.
- 2 Rows are de-duplicated against existing trade ids.
- 3 `POST /api/trades/bulk` with `{ trades: [...] }` → `Trade.insertMany(docs, { ordered: false })`.
- 4 Fresh trades appear in the list; count returned to the user.

8.5 Listing & filtering

`GET /api/trades` supports query filters: `search` (regex on pair), `direction`, `result`, `startDate` / `endDate`, and whitelisted `sort` fields (e.g. `-createdAt`).

9. API Reference

Method & Path	Auth	Description
<code>GET /api/health</code>	No	Liveness check <code>{ status: 'ok', timestamp }</code>
<code>POST /api/auth/register</code>	No	Create account → <code>201</code> with token + user
<code>POST /api/auth/login</code>	No	Login → token + user (<code>401</code> on bad credentials)
<code>GET /api/auth/me</code>	JWT	Current user profile
<code>PUT /api/auth/profile</code>	JWT	Update <code>name</code>
<code>PUT /api/auth/password</code>	JWT	Change password (verifies current)
<code>GET /api/trades</code>	JWT	List/filter user's trades
<code>POST /api/trades</code>	JWT	Create one trade
<code>DELETE /api/trades</code>	JWT	Clear all user's trades
<code>POST /api/trades/bulk</code>	JWT	Bulk import trades
<code>GET /api/trades/:id</code>	JWT	Get one trade
<code>PUT /api/trades/:id</code>	JWT	Update one trade
<code>DELETE /api/trades/:id</code>	JWT	Delete one trade

10. Data Models

User (`backend/models/User.js`)

Field	Type	Notes
<code>name</code>	String	required, trimmed

email	String	required, unique, lowercase, validated by regex
passwordHash	String	bcrypt hash (10 rounds); never serialized to JSON
createdAt/updatedAt	Date	auto via timestamps: true

Trade (backend/models/Trade.js)

Field	Type	Notes
_id	String	default crypto.randomUUID() — keeps CSV round-trip ids
user	ObjectId ref → User	required, indexed — ownership + isolation
date	String YYYY-MM-DD	required
entryTime	String	optional
pair	String	required, uppercased
session	enum	London / NewYork / Overlap / Other (default Other)
direction	enum	Buy / Sell (required)
zoneType	enum	Supply / Demand
entry / stopLoss / target	Number > 0	required, validated positive
result	enum	Win / Loss / BreakEven / Open
rr	Number ≥ 0	risk/reward ratio
notes / emotion / reason / screenshot	String	optional
tags	[String]	default []
mindset	{ before, after }	optional strings
isValidRuleTrade	Boolean	default false

11. Frontend Pages & What They Do

Route	Page	Purpose
/login	Login	Login / register form, “remember me”
/	Dashboard	Stat cards (trades, win rate, P&L, RR), recent trades
/log	TradeLog	Full trade table with search/filter, bulk select, delete, export
/add, /edit/:id	AddEditTrade	Trade form (create / edit), duplicate
/analytics	AnalyticsPage	Equity curve, candle chart, stats via Recharts
/checklist	RulesChecklist	Trading rules checklist (stored locally per browser)
/backtest	BacktestProgress	Backtest goal tracking
/backtest/chart	ChartBacktest	Chart-based backtest tool
/profile	Profile	Edit name, change password, export/backup data

Protected routes redirect to /login when no user session exists (see App.tsx ProtectedRoute).

12. What Is Stored Where

Data	Location	Why
Users + trades	MongoDB (backend) — source of truth	Server-side, per-user isolation, survives browser clears
JWT token	localStorage or sessionStorage	Remember-me decides which; API client attaches it
Theme (dark/light)	localStorage	UI preference
Trading rules checklist	localStorage	Browser-local preference (not yet migrated to DB)

	(<code>tradelog.rules.*</code>)	
Backtest goal settings, backup reminder counters	<code>localStorage</code>	Browser-local settings

13. Startup Flow — How Everything Boots

- 1
- Backend** — `cd backend && npm run dev` (nodemon → `server.js`).
- 2
- `dotenv` loads `backend/.env` ; missing `JWT_SECRET` aborts startup.
- 3
- `connectDB()` connects Mongoose to `MONGO_URI` ; failure aborts startup.
- 4
- Express starts on port `5000` . Routes mounted under `/api` . 404 and error handlers last.
- 5
- Frontend** — `cd frontend && npm run dev` starts Vite on port `5173` .
- 6
- Vite serves the SPA and proxies `/api/*` → `localhost:5000` ; same-origin calls need no CORS in dev.
- 7
- Browser loads `index.html` → React mounts → providers initialize → session restore → page renders.

14. Running Locally / on Atlas

```
# Backend
cd backend
npm install
cp .env.example .env          # then set MONGO_URI and JWT_SECRET
npm run dev                   # http://localhost:5000

# Frontend
cd frontend
npm install
npm run dev                   # http://localhost:5173
```

`VITE_API_URL` is optional; when unset the client uses `/api` , which works through the Vite proxy in development.

Environment variables (`backend/.env.example`):

Variable	Purpose
<code>NODE_ENV</code>	development / production
<code>PORT</code>	API port (default 5000)
<code>MONGO_URI</code>	Local: <code>mongodb://127.0.0.1:27017/tradelog</code> Atlas: <code>mongodb+srv://<user>:<pass>@<cluster>.mongodb.net/tradelog?retryWrites=true&w=majority</code>
<code>CORS_ORIGIN</code>	Comma-separated allowed origins (default: all)
<code>JWT_SECRET</code>	JWT signing secret (required)
<code>JWT_EXPIRES_IN</code>	Token lifetime (default 7d)

15. MongoDB — Local vs Atlas (current status)

Current environment: this sandbox preview runs against a **local MongoDB** (`mongodb://127.0.0.1:27017/tradelog` , MongoDB 8). The Atlas connection string is already prepared in `backend/.env` and `backend/.env.example` , ready to switch.

To move to **MongoDB Atlas**:

1. Set `MONGO_URI` in `backend/.env` to your Atlas cluster string.
2. Whitelist your machine’s public IP under Atlas → Network Access (already done for `27.34.73.7/32`).
3. Restart the backend; new accounts and trades are then stored on Atlas.

Note: data written to the sandbox’s local MongoDB does not automatically move to Atlas. Export your trades (CSV) in the app and re-import them after registering on Atlas, or start fresh. The current `backend/.env` is gitignored and keeps the sandbox-local URI active.

16. Build & Quality Checks

Command	What it does
<code>cd frontend && npm run build</code>	TypeScript project build (<code>tsc -b</code>) + Vite production build
<code>cd frontend && npm run lint</code>	ESLint over <code>src/</code> (typescript-eslint + react-hooks rules)
<code>cd backend && npm start</code>	Production-style backend start

Verified in this workspace: build and lint both exit 0; the API was end-to-end tested through the Vite proxy (register, login, me, duplicate-email 409, wrong-password 401, create/list/update/delete/ bulk/clear-all, change password, and cross-user isolation).

17. Repository & Git

- Remote: `https://github.com/nabingiri99/tradelog` (branch `main`).
- `backend/.env` is **gitignored** — secrets (JWT_SECRET, Atlas password) never enter the repository; only the `.env.example` template is committed.
- Commits follow the message style used in the repo (e.g. `feat: connect frontend to backend with JWT auth`).

18. End-to-End Scenario Walkthrough

A complete example: a trader opens the app, signs up, records a trade, reviews analytics, and backs up data.

Scenario A — First run

1. Open the frontend (Vite dev server, port 5173).
2. No stored token → `AuthProvider.restoring` resolves immediately, `user = null`, `ProtectedRoute` redirects to `/login`.
3. Trader fills the register form. `AuthProvider.register()` → `POST /api/auth/register`.
4. Backend hashes password, creates `User`, returns `201` + JWT.
5. Token stored, `user` set. App re-renders and `TradeProvider` fetches `GET /api/trades` (empty list for a new user).
6. Dashboard renders with zeroed stats.

Scenario B — Log a trade

1. Trader opens `/add`, fills the form (pair, direction, entry, stop loss, target, result, notes, tags, psychology).
2. Submit → `TradeProvider.addTrade()`: a UUID is generated, the table shows the trade instantly (optimistic).
3. `POST /api/trades` persists it under `user: req.user._id`.
4. Confirmed record replaces the optimistic row. Stats recompute from `trades`.

Scenario C — Review performance

1. Trader opens `/analytics`.
2. `analytics.ts` computes win rate, average RR, profit/loss from the current `trades` state.
3. `Recharts` renders the equity curve and candle chart from the same array.

Scenario D — Backup / migrate

1. In `/log` or Profile, the trader exports a CSV (`tradesToCsv`) containing trade ids.
2. `BackupReminder` counts trades logged since the last export and nudges the trader to export.
3. To migrate to Atlas: register a new account there, then import the CSV → `parseCsv` → `POST /api/trades/bulk` → trades restored with their original ids.

19. Security & Reliability Summary

- **AuthN:** JWT (HS256, signed with `JWT_SECRET`), verified on every protected route.
- **AuthZ:** all trade queries filter by `user`; cross-user access returns 404.
- **Passwords:** bcrypt hashed, never returned by the API.
- **Headers:** `helmet`; CORS restricted to configured origins.
- **Input:** schema enums + positive-number validation at the middleware and Mongoose levels.
- **Resilience:** centralized error handler maps Mongoose errors (validation, cast, duplicate key) to clean HTTP responses; async errors are caught by `asyncHandler`.
- **Frontend:** optimistic updates roll back on failure; bulk operations are de-duplicated; 401 auto-clears the session.

20. Possible Next Steps

- Run on your own machine against MongoDB Atlas (uncomment the Atlas URI in `backend/.env`).
- Migrate the trading-rules checklist and backtest-goal settings from `localStorage` into MongoDB so they sync across devices.
- Add JWT refresh tokens, email verification, or password reset.
- Add automated tests (unit + API integration) and CI.
- Deploy backend and frontend to a hosting provider; serve the built frontend statically behind the API.

Generated for reference — TradeLog. This document reflects the code committed to `github.com/nabingiri99/tradelog` (commit `809ec93`).